Max Sum Increasing Subsequence

AlgoExpert · Hard · Dynamic Programming

The problem

- Returns [[maxSum], [subsequence]] for the strictly increasing subsequence with the greatest sum.

Intuition

- For each index, the best sum ending there extends the best smaller earlier element.

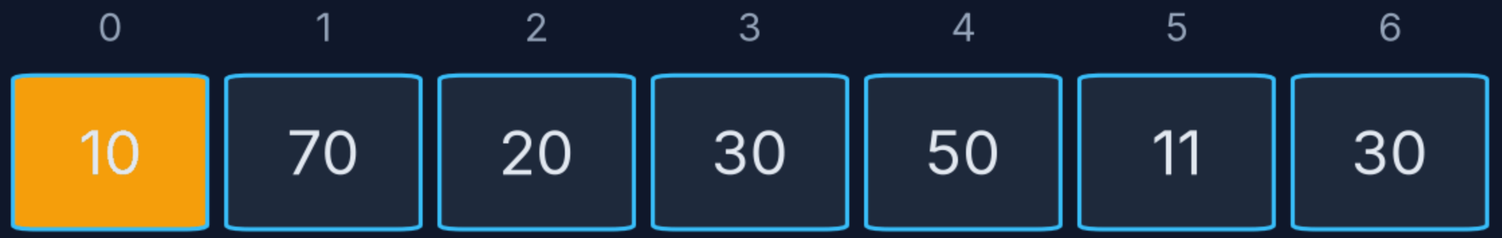
Brute force vs optimal

- All increasing subsequences → exponential
- LIS-style DP on sums → $O(n^2)$
- Prev[] to rebuild

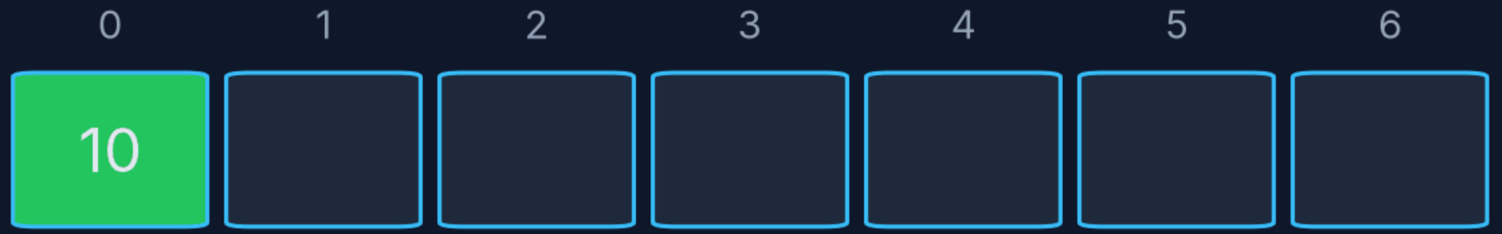
Approach

- 1. For each i, scan earlier j with smaller value
- 2. Keep best sums and predecessors
- 3. Rebuild the sequence from the best end

Worked example

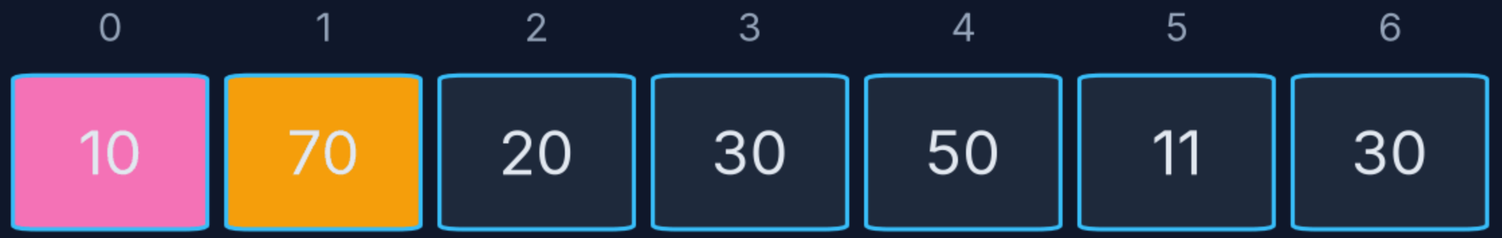


best sum ending here

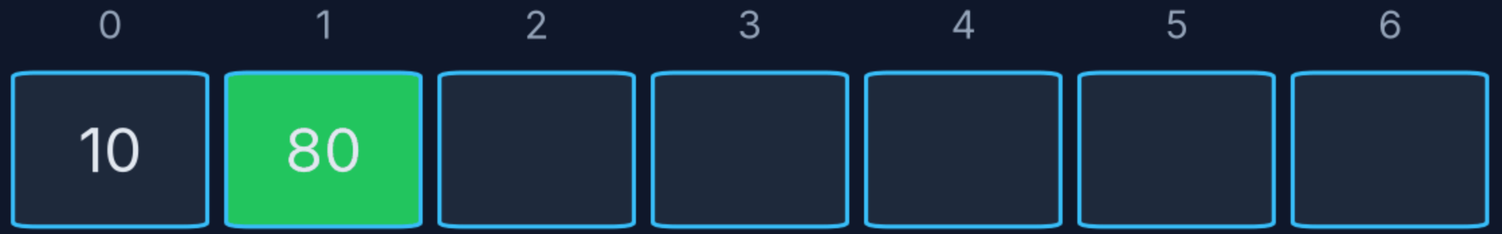


10: best sum ending here = 10

Worked example

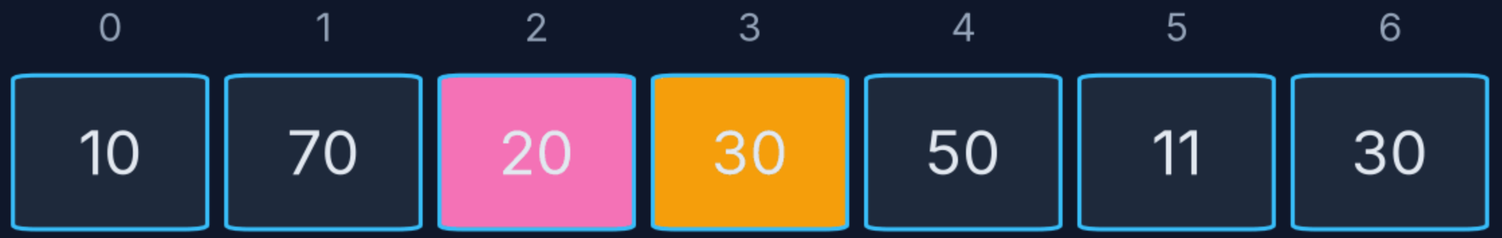


best sum ending here

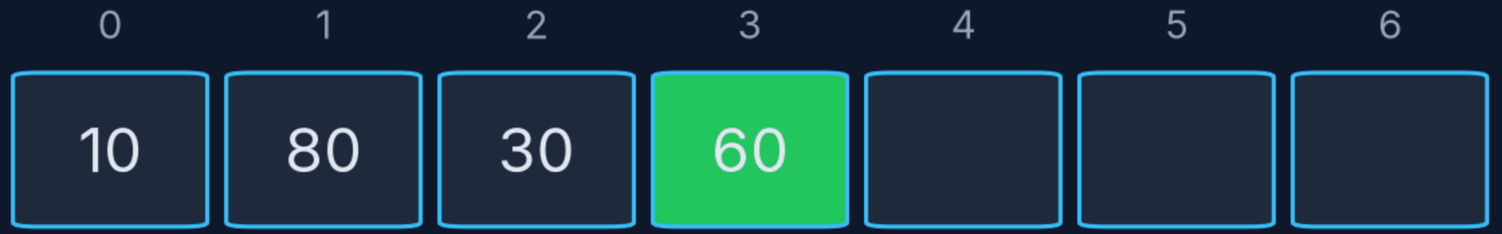


70: best sum ending here = 80

Worked example

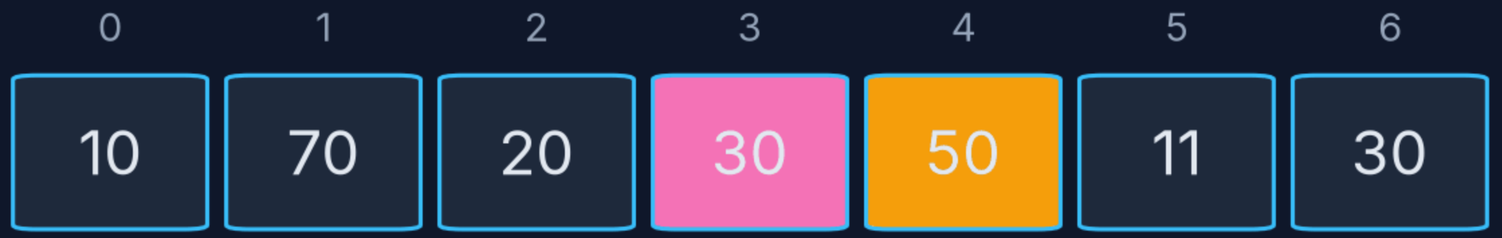


best sum ending here

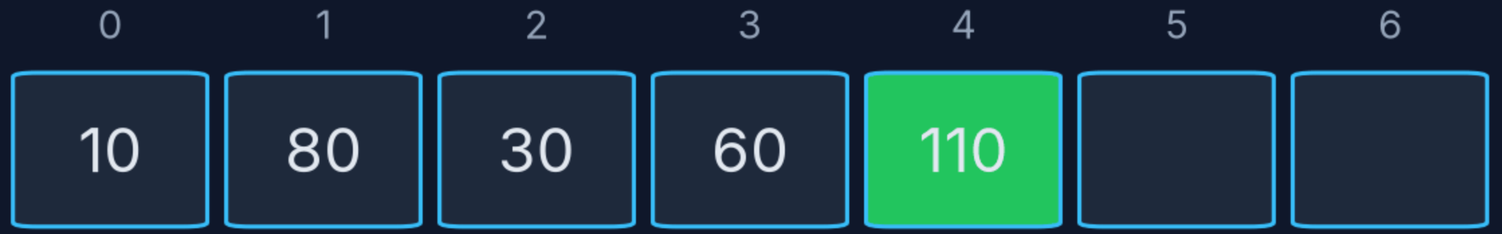


30: best sum ending here = 60

Worked example

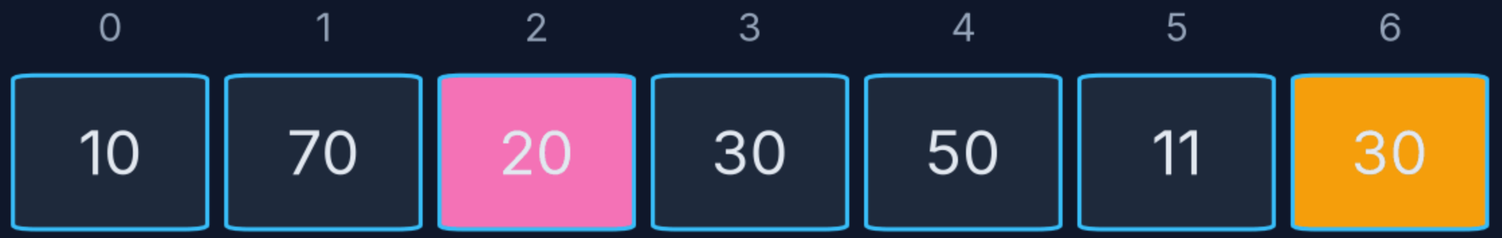


best sum ending here

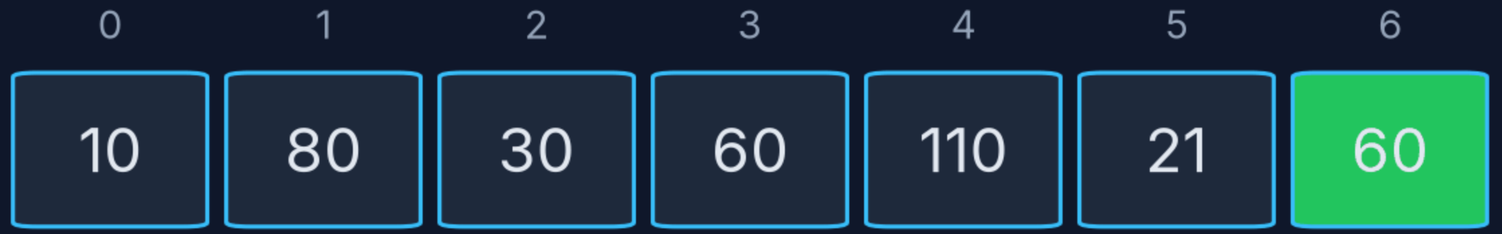


50: best sum ending here = 110

Worked example



best sum ending here



30: best sum ending here = 60

Worked example

```
sum 110: [10, 20, 30, 50]
```

Answer: 110 via [10, 20, 30, 50]

Java solution

```java
public class MaxSumIncreasingSubsequence {
    public static List<List<Integer>> maxSumIncreasingSubsequence(int[] a) {
        int n = a.length, bestIdx = 0;
        int[] sums = a.clone(), prev = new int[n];
        for (int i = 0; i < n; i++) {
            prev[i] = -1;
            for (int j = 0; j < i; j++)
                if (a[j] < a[i] && sums[j] + a[i] > sums[i]) { sums[i] = sums[j] + a[i]; prev[i] = j; }
            if (sums[i] > sums[bestIdx]) bestIdx = i;
        }
        List<Integer> seq = new ArrayList<>();
        for (int i = bestIdx; i != -1; i = prev[i]) seq.add(a[i]);
        Collections.reverse(seq);
        return List.of(List.of(sums[bestIdx]), seq);
    }
}
```

Complexity

- Time: O(n^2)
- Space: O(n)

Edge cases & pitfalls

- Strictly increasing
- Negatives: single element might win
- Track prev

Interview tips & follow-ups

- Follow-up: O(n log n) with a BIT keyed by value

Recap

- For each index, the best sum ending there extends the best smaller earlier element.
- Complexity: O(n^2)